ASSIGNMENT 9

Advanced SQL Queries Using Joins and Subqueries

Multi-table business reports for the Online Shopping System

1. Objective

Real reports almost always need data from several tables at once. This assignment builds twenty-one reports that combine tables with joins, compare rows with subqueries, and summarise the result. Reports are grouped as: **A. Join reports**, **B. Subquery reports** and **C. Combined and summary reports**. Each shows the SQL, the output and a short explanation.

A. Join Reports

Report 1 – Order invoice lines (INNER JOIN of four tables)

SQL query

```
SELECT o.order_id, CONCAT(c.first_name,' ',c.last_name) AS customer,
       LEFT(p.product_name,26) AS product, oi.quantity AS qty, oi.unit_price,
       oi.quantity * oi.unit price AS line total
FROM Orders o
JOIN Customers   c  ON c.customer_id = o.customer_id
JOIN Order Items oi ON oi.order id   = o.order id
JOIN Products    p  ON p.product id  = oi.product id
WHERE o.order_id IN (6, 8)
ORDER BY o.order_id, p.product_name;
```

Output

```
+----------+--------------+----------------------------+-----+------------+------------+
| order id | customer     | product                    | qty | unit price | line total |
+----------+--------------+----------------------------+-----+------------+------------+
|        6 | Aarav Sharma | Digital Air Fryer 4.1L     |   1 |    7999.00 |    7999.00 |
|        6 | Aarav Sharma | Leather Formal Shoes       |   1 |    2299.00 |    2299.00 |
|        6 | Aarav Sharma | Women's Cotton Kurta       |   1 |    1899.00 |    1899.00 |
|        6 | Aarav Sharma | Yoga Mat 6mm               |   3 |     549.00 |    1647.00 |
|        8 | Rohan Mehta  | Atomic Habits              |   3 |     499.00 |    1497.00 |
|        8 | Rohan Mehta  | Men's Formal Shirt         |   1 |    1499.00 |    1499.00 |
|        8 | Rohan Mehta  | Revolution 7 Running Shoes |   1 |    3995.00 |    3995.00 |
+----------+--------------+----------------------------+-----+------------+------------+
```

Explanation: Four tables are linked through their foreign keys to produce a readable invoice: who ordered, what, how many and at what price.

Report 2 – Delivery address of every shipped or delivered order (joins through Addresses and Pincodes)

SQL query

```
SELECT o.order id, CONCAT(c.first name,' ',c.last name) AS customer,
       a.address type, p.city, p.state, o.order status
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Addresses a ON a.address id  = o.address id
JOIN Pincodes  p ON p.pincode     = a.pincode
WHERE o.order_status IN ('Shipped','Placed')
ORDER BY o.order_id;
```

Output

```
+----------+---------------+--------------+-----------+-------------+--------------+
| order_id | customer      | address_type | city      | state       | order_status |
+----------+---------------+--------------+-----------+-------------+--------------+
|       21 | Pooja Bansal  | Home         | New Delhi | Delhi       | Shipped      |
|       22 | Rohan Mehta   | Home         | Mumbai    | Maharashtra | Shipped      |
|       23 | Vikram Singh  | Home         | New Delhi | Delhi       | Shipped      |
|       24 | Meera Joshi   | Home         | Pune      | Maharashtra | Shipped      |
|       25 | Sneha Reddy   | Home         | Hyderabad | Telangana   | Placed       |
|       26 | Karthik Rao   | Work         | Bengaluru | Karnataka   | Placed       |
|       27 | Siddharth Das | Home         | Kolkata   | West Bengal | Placed       |
|       28 | Karthik Rao   | Home         | Bengaluru | Karnataka   | Placed       |
+----------+---------------+--------------+-----------+-------------+--------------+
```

Explanation: City and state come from the Pincodes table that was introduced in Assignment 3 – the normalized design costs one extra join but removes redundancy.

Report 3 – Customers who have never placed an order (LEFT JOIN … IS NULL)

SQL query

```
SELECT c.customer id, CONCAT(c.first name,' ',c.last name) AS customer, c.registration date
FROM Customers c
LEFT JOIN Orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

Output

```
+-------------+-------------------+-------------------+
| customer id | customer          | registration date |
+-------------+-------------------+-------------------+
|          11 | Imran Khan        | 2025-08-19        |
|          15 | Rahul Verma       | 2025-11-04        |
|          16 | Lakshmi Narayanan | 2025-12-01        |
+-------------+-------------------+-------------------+
```

Explanation: A LEFT JOIN keeps every customer; those with no matching order get NULLs, which the WHERE clause isolates – a list for a re-engagement campaign.

Report 4 – Products that have never been ordered

SQL query

```
SELECT p.product id, LEFT(p.product name,30) AS product, cat.category name, p.price, p.stock quantity AS stock
FROM Products p
JOIN Categories cat ON cat.category_id = p.category_id
LEFT JOIN Order_Items oi ON oi.product_id = p.product_id
WHERE oi.order item id IS NULL
ORDER BY cat.category_name, p.product_name;
```

Output

```
+------------+--------------------------------+------------------+---------+-------+
| product id | product                        | category name    | price   | stock |
+------------+--------------------------------+------------------+---------+-------+
|          6 | 511 Slim Fit Jeans             | Clothing         | 2799.00 |    60 |
|         16 | Glass Tumbler Set (6 pcs)      | Home & Kitchen   |  699.00 |    80 |
|         22 | Mavis 350 Nylon Shuttlecocks ( | Sports & Fitness |  649.00 |   100 |
+------------+--------------------------------+------------------+---------+-------+
```

Explanation: Slow-moving stock that may need promotion or a price review.

Report 5 – Revenue by category (excluding cancelled orders)

SQL query

```
SELECT cat.category_name, SUM(oi.quantity) AS units_sold,
       SUM(oi.quantity * oi.unit price) AS revenue,
       ROUND(100 * SUM(oi.quantity * oi.unit_price) /
             (SELECT SUM(oi2.quantity * oi2.unit_price) FROM Order_Items oi2
              JOIN Orders o2 ON o2.order_id = oi2.order_id
              WHERE o2.order status <> 'Cancelled'), 1) AS pct of total
FROM Order_Items oi
JOIN Orders     o   ON o.order_id     = oi.order_id
JOIN Products   p   ON p.product id   = oi.product id
JOIN Categories cat ON cat.category id = p.category id
WHERE o.order_status <> 'Cancelled'
GROUP BY cat.category name
ORDER BY revenue DESC;
```

Output

```
+------------------+------------+-----------+--------------+
| category_name    | units_sold | revenue   | pct_of_total |
+------------------+------------+-----------+--------------+
| Electronics      |         26 | 140506.00 |         64.5 |
| Footwear         |         12 |  32668.00 |         15.0 |
| Home & Kitchen   |          7 |  17893.00 |          8.2 |
| Clothing         |          8 |  10292.00 |          4.7 |
| Sports & Fitness |         13 |  10287.00 |          4.7 |
| Books            |         14 |   6236.00 |          2.9 |
+------------------+------------+-----------+--------------+
```

Explanation: Four joins plus GROUP BY give revenue per category; the scalar subquery in the SELECT list supplies the grand total used to compute each category's share.

Report 6 – Sales by city

SQL query

```
SELECT p.city, p.state, COUNT(DISTINCT o.order id) AS orders,
       COUNT(DISTINCT o.customer_id) AS customers, SUM(o.total_amount) AS sales
FROM Orders o
JOIN Addresses a ON a.address id = o.address id
JOIN Pincodes  p ON p.pincode    = a.pincode
WHERE o.order_status <> 'Cancelled'
```

```
GROUP BY p.city, p.state
ORDER BY sales DESC;
```

Output

```
+-----------+-------------+--------+-----------+-----------+
| city      | state       | orders | customers | sales     |
+-----------+-------------+--------+-----------+-----------+
| Bengaluru | Karnataka   |      9 |         4 | 100676.00 |
| Pune      | Maharashtra |      5 |         2 |  54110.00 |
| Mumbai    | Maharashtra |      4 |         1 |  36723.00 |
| New Delhi | Delhi       |      2 |         2 |   8990.00 |
| Kolkata   | West Bengal |      2 |         1 |   7493.00 |
| Ahmedabad | Gujarat     |      2 |         1 |   7391.00 |
| Hyderabad | Telangana   |      1 |         1 |   2499.00 |
+-----------+-------------+--------+-----------+-----------+
```

Explanation: Geographic demand report for logistics and marketing, based on the delivery address of each order.

Report 7 – Courier performance

SQL query

```
SELECT s.courier_name, COUNT(*) AS shipments,
       SUM(s.shipment_status = 'Delivered') AS delivered_n,
       SUM(s.shipment status = 'In Transit') AS in transit n,
       SUM(s.shipment status = 'Delayed')    AS delayed n,
       ROUND(AVG(DATEDIFF(s.expected_delivery_date, s.shipped_date)),1) AS avg_promised_days
FROM Shipments s
JOIN Orders o ON o.order id = s.order id
GROUP BY s.courier_name
ORDER BY shipments DESC;
```

Output

```
+--------------+-----------+-------------+--------------+-----------+--------------------+
| courier name | shipments | delivered n | in transit n | delayed n | avg promised days  |
+--------------+-----------+-------------+--------------+-----------+--------------------+
| Blue Dart    |        10 |           6 |            3 |         1 |                3.7 |
| XpressBees   |         5 |           5 |            0 |         0 |                4.4 |
| Ecom Express |         3 |           3 |            0 |         0 |                4.7 |
| India Post   |         3 |           3 |            0 |         0 |                4.3 |
+--------------+-----------+-------------+--------------+-----------+--------------------+
```

Explanation: Conditional counting with SUM(condition) turns status values into columns; DATEDIFF measures the delivery time each courier promised.

Report 8 – Payments still outstanding on active orders

SQL query

```
SELECT o.order id, CONCAT(c.first name,' ',c.last name) AS customer, o.order status,
       pay.payment_method, pay.payment_status, o.total_amount AS order_total, pay.amount_paid
FROM Orders o
JOIN Customers c   ON c.customer id = o.customer id
JOIN Payments  pay ON pay.order id  = o.order id
WHERE o.order_status IN ('Placed','Shipped') AND pay.payment_status <> 'Successful'
ORDER BY o.order_id;
```

Output

```
+----------+---------------+--------------+------------------+----------------+-------------+-------------+
| order id | customer      | order status | payment method   | payment status | order total | amount paid |
+----------+---------------+--------------+------------------+----------------+-------------+-------------+
|       25 | Sneha Reddy   | Placed       | Cash on Delivery | Pending        |     2499.00 |        0.00 |
|       27 | Siddharth Das | Placed       | Cash on Delivery | Pending        |     5994.00 |        0.00 |
+----------+---------------+--------------+------------------+----------------+-------------+-------------+
```

Explanation: Joining Orders to Payments reveals active orders that are not yet paid – all of them Cash on Delivery, to be collected on delivery.

Report 9 – Product scorecard: sales and ratings together (LEFT JOIN to two derived tables)

SQL query

```
SELECT p.product id AS id, LEFT(p.product name,28) AS product,
       COALESCE(s.units,0) AS units sold, COALESCE(r.reviews,0) AS reviews, r.avg rating
FROM Products p
LEFT JOIN (SELECT oi.product_id, SUM(oi.quantity) AS units
           FROM Order Items oi JOIN Orders o ON o.order id = oi.order id
           WHERE o.order status <> 'Cancelled' GROUP BY oi.product id) s ON s.product id = p.product id
LEFT JOIN (SELECT product id, COUNT(*) AS reviews, ROUND(AVG(rating),2) AS avg rating
           FROM Reviews GROUP BY product id) r ON r.product id = p.product id
ORDER BY units_sold DESC, p.product_id
LIMIT 12;
```

Output

```
+----+------------------------------+------------+---------+------------+
```

```
| id | product                       | units_sold | reviews | avg_rating |
+----+-------------------------------+------------+---------+------------+
|  4 | M331 Silent Plus Wireless Mo |          7 |       1 |       4.00 |
| 21 | Wings of Fire                 |          7 |       1 |       5.00 |
|  1 | Redmi Note 13 5G (8GB/128GB) |          6 |       3 |       4.67 |
|  2 | Rockerz 450 Bluetooth Headph |          6 |       2 |       4.50 |
| 24 | Yoga Mat 6mm                  |          6 |       2 |       3.50 |
| 10 | Revolution 7 Running Shoes    |          5 |       3 |       3.67 |
|  3 | 25W Fast Charger              |          4 |       1 |       4.00 |
|  7 | Men's Formal Shirt            |          4 |       2 |       4.00 |
| 18 | Atomic Habits                 |          4 |       1 |       5.00 |
| 25 | PVC Dumbbell Set 10 kg        |          4 |       1 |       2.00 |
|  5 | ColorFit Pulse 3 Smartwatch   |          3 |       0 |       NULL |
|  8 | Round Neck Cotton T-Shirt     |          3 |       1 |       5.00 |
+----+-------------------------------+------------+---------+------------+
```

Explanation: Sales and reviews are aggregated separately in derived tables and then joined, which avoids the double counting that would occur if Order_Items and Reviews were joined directly. COALESCE turns missing values into 0.

Report 10 – Cart value per customer (including empty carts)

SQL query

```
SELECT c.customer id, CONCAT(c.first name,' ',c.last name) AS customer,
       COUNT(ci.cart_item_id) AS items, COALESCE(SUM(ci.quantity * p.price),0) AS cart_value
FROM Customers c
JOIN Shopping Cart sc ON sc.customer id = c.customer id
LEFT JOIN Cart Items ci ON ci.cart id = sc.cart id
LEFT JOIN Products p ON p.product_id = ci.product_id
GROUP BY c.customer id, customer
ORDER BY cart_value DESC;
```

Output

```
+-------------+-------------------+-------+------------+
| customer_id | customer          | items | cart_value |
+-------------+-------------------+-------+------------+
|           1 | Aarav Sharma      |     3 |   17396.00 |
|           5 | Vikram Singh      |     2 |    9596.00 |
|          12 | Pooja Bansal      |     3 |    8594.00 |
|           8 | Meera Joshi       |     3 |    8096.00 |
|           6 | Sneha Reddy       |     2 |    5147.00 |
|           3 | Rohan Mehta       |     2 |    5097.00 |
|          10 | Divya Menon       |     2 |    4897.00 |
|          13 | Siddharth Das     |     2 |    4398.00 |
|           2 | Priya Nair        |     3 |    3347.00 |
|          14 | Neha Kulkarni     |     2 |    2698.00 |
|           9 | Arjun Patel       |     0 |       0.00 |
|          16 | Lakshmi Narayanan |     0 |       0.00 |
|          15 | Rahul Verma       |     0 |       0.00 |
|           4 | Ananya Iyer       |     0 |       0.00 |
|          11 | Imran Khan        |     0 |       0.00 |
|           7 | Karthik Rao       |     0 |       0.00 |
+-------------+-------------------+-------+------------+
```

Explanation: The LEFT JOINs keep customers whose cart is empty (value 0). Cart items are priced at the *current* product price, unlike order lines which keep the price paid.

Report 11 – SELF JOIN: pairs of customers living in the same city

SQL query

```
SELECT DISTINCT p.city, CONCAT(c1.first name,' ',c1.last name) AS customer 1,
       CONCAT(c2.first name,' ',c2.last name) AS customer 2
FROM Addresses a1
JOIN Addresses a2 ON a1.pincode <> '' AND a1.customer_id < a2.customer_id
JOIN Pincodes p1 ON p1.pincode = a1.pincode
JOIN Pincodes p  ON p.pincode  = a2.pincode AND p.city = p1.city
JOIN Customers c1 ON c1.customer_id = a1.customer_id
JOIN Customers c2 ON c2.customer_id = a2.customer_id
ORDER BY p.city, customer 1, customer 2
LIMIT 12;
```

Output

```
+-----------+--------------+--------------------+
| city      | customer_1   | customer_2         |
+-----------+--------------+--------------------+
| Bengaluru | Aarav Sharma | Divya Menon        |
| Bengaluru | Aarav Sharma | Karthik Rao        |
| Bengaluru | Aarav Sharma | Priya Nair         |
| Bengaluru | Karthik Rao  | Divya Menon        |
| Bengaluru | Priya Nair   | Divya Menon        |
| Bengaluru | Priya Nair   | Karthik Rao        |
| Chennai   | Ananya Iyer  | Lakshmi Narayanan  |
| Hyderabad | Sneha Reddy  | Rahul Verma        |
| Mumbai    | Rohan Mehta  | Imran Khan         |
| New Delhi | Vikram Singh | Pooja Bansal       |
| Pune      | Meera Joshi  | Neha Kulkarni      |
+-----------+--------------+--------------------+
```

Explanation: Addresses is joined to itself (with a1.customer_id < a2.customer_id to avoid mirrored duplicates) to find neighbours – useful for local delivery batching.

B. Subquery Reports

Report 12 – Scalar subquery: products priced above the overall average

SQL query

```
SELECT product id, LEFT(product name,30) AS product, price,
       (SELECT ROUND(AVG(price),2) FROM Products) AS overall_avg
FROM Products
WHERE price > (SELECT AVG(price) FROM Products)
ORDER BY price DESC;
```

Output

```
+------------+------------------------------+----------+-------------+
| product_id | product                      | price    | overall_avg |
+------------+------------------------------+----------+-------------+
|          1 | Redmi Note 13 5G (8GB/128GB) | 17999.00 |     2292.84 |
|         17 | Digital Air Fryer 4.1L       |  7999.00 |     2292.84 |
|         10 | Revolution 7 Running Shoes   |  3995.00 |     2292.84 |
|          6 | 511 Slim Fit Jeans           |  2799.00 |     2292.84 |
|         14 | Induction Cooktop 2000W      |  2499.00 |     2292.84 |
|         11 | Leather Formal Shoes         |  2299.00 |     2292.84 |
+------------+------------------------------+----------+-------------+
```

Explanation: The inner query returns a single number (the average price) which the outer query uses as a threshold.

Report 13 – IN subquery: customers who bought at least one Electronics product

SQL query

```
SELECT customer_id, CONCAT(first_name,' ',last_name) AS customer, email
FROM Customers
WHERE customer id IN (SELECT o.customer id
                      FROM Orders o
                      JOIN Order_Items oi ON oi.order_id = o.order_id
                      JOIN Products p     ON p.product_id = oi.product_id
                      WHERE p.category id = 1 AND o.order status <> 'Cancelled')
ORDER BY customer_id;
```

Output

```
+-------------+---------------+--------------------------+
| customer_id | customer      | email                    |
+-------------+---------------+--------------------------+
|           2 | Priya Nair    | priya.nair@yahoo.in      |
|           3 | Rohan Mehta   | rohan.mehta@gmail.com    |
|           7 | Karthik Rao   | karthik.rao@gmail.com    |
|           8 | Meera Joshi   | meera.joshi@gmail.com    |
|           9 | Arjun Patel   | arjun.patel@outlook.com  |
|          10 | Divya Menon   | divya.menon@yahoo.in     |
|          12 | Pooja Bansal  | pooja.bansal@outlook.com |
|          14 | Neha Kulkarni | neha.kulkarni@gmail.com  |
+-------------+---------------+--------------------------+
```

Explanation: The subquery produces a list of customer ids; the outer query returns their details – a ready-made target list for an electronics promotion.

Report 14 – Subquery in HAVING: customers who spent more than the average customer

SQL query

```
SELECT o.customer id, CONCAT(c.first name,' ',c.last name) AS customer, SUM(o.total amount) AS total spent
FROM Orders o JOIN Customers c ON c.customer_id = o.customer_id
WHERE o.order_status <> 'Cancelled'
GROUP BY o.customer id, customer
HAVING SUM(o.total amount) > (SELECT AVG(t.spent)
                              FROM (SELECT SUM(total amount) AS spent FROM Orders
                                    WHERE order status <> 'Cancelled' GROUP BY customer id) t)
ORDER BY total_spent DESC;
```

Output

```
+-------------+---------------+-------------+
| customer id | customer      | total spent |
+-------------+---------------+-------------+
|          14 | Neha Kulkarni |    52611.00 |
|           3 | Rohan Mehta   |    36723.00 |
|           7 | Karthik Rao   |    35833.00 |
|          10 | Divya Menon   |    31560.00 |
|           2 | Priya Nair    |    19439.00 |
+-------------+---------------+-------------+
```

Explanation: A derived table computes each ordering customer's spend, the outermost subquery averages it, and HAVING keeps only above-average customers – the “VIP” list.

Report 15 – Correlated subquery: each customer's most recent order

SQL query

```
SELECT o.customer id, CONCAT(c.first name,' ',c.last name) AS customer,
       o.order id, o.order date, o.order status, o.total amount
FROM Orders o JOIN Customers c ON c.customer_id = o.customer_id
WHERE o.order date = (SELECT MAX(o2.order date) FROM Orders o2 WHERE o2.customer id = o.customer id)
ORDER BY o.order_date DESC;
```

Output

```
+-------------+---------------+----------+---------------------+--------------+--------------+
| customer_id | customer      | order_id | order_date          | order_status | total_amount |
+-------------+---------------+----------+---------------------+--------------+--------------+
|           7 | Karthik Rao   |       28 | 2026-10-20 15:32:00 | Placed       |      5194.00 |
|          13 | Siddharth Das |       27 | 2026-10-19 19:31:00 | Placed       |      5994.00 |
|           6 | Sneha Reddy   |       25 | 2026-10-14 15:23:00 | Placed       |      2499.00 |
|           8 | Meera Joshi   |       24 | 2026-10-12 20:33:00 | Shipped      |      1499.00 |
|           5 | Vikram Singh  |       23 | 2026-10-10 17:29:00 | Shipped      |      1499.00 |
|           3 | Rohan Mehta   |       22 | 2026-10-09 18:56:00 | Shipped      |     22297.00 |
|          12 | Pooja Bansal  |       21 | 2026-10-08 08:09:00 | Shipped      |      7491.00 |
|          14 | Neha Kulkarni |       19 | 2026-09-21 13:18:00 | Delivered    |      6142.00 |
|           4 | Ananya Iyer   |       18 | 2026-09-17 16:27:00 | Cancelled    |      1647.00 |
|           9 | Arjun Patel   |       17 | 2026-09-13 21:30:00 | Delivered    |      4794.00 |
|           2 | Priya Nair    |        9 | 2026-07-12 21:29:00 | Delivered    |     19439.00 |
|           1 | Aarav Sharma  |        6 | 2026-05-17 12:30:00 | Delivered    |     13844.00 |
|          10 | Divya Menon   |        3 | 2026-03-05 20:06:00 | Delivered    |      2590.00 |
+-------------+---------------+----------+---------------------+--------------+--------------+
```

Explanation: The inner query refers to the outer row (o.customer_id), so it is re-evaluated per customer to find that customer's latest order date.

Report 16 – Correlated subquery: products priced above their own category's average

SQL query

```
SELECT p.category id, LEFT(p.product name,30) AS product, p.price,
       (SELECT ROUND(AVG(p2.price),2) FROM Products p2 WHERE p2.category id = p.category id) AS category avg
FROM Products p
WHERE p.price > (SELECT AVG(p2.price) FROM Products p2 WHERE p2.category_id = p.category_id)
ORDER BY p.category_id, p.price DESC;
```

Output

```
+-------------+------------------------------+----------+--------------+
| category_id | product                      | price    | category_avg |
+-------------+------------------------------+----------+--------------+
|           1 | Redmi Note 13 5G (8GB/128GB) | 17999.00 |      4719.00 |
|           2 | 511 Slim Fit Jeans           |  2799.00 |      1749.00 |
|           2 | Women's Cotton Kurta         |  1899.00 |      1749.00 |
|           3 | Revolution 7 Running Shoes   |  3995.00 |      2398.00 |
|           4 | Digital Air Fryer 4.1L       |  7999.00 |      2999.00 |
|           5 | Database System Concepts     |   899.00 |       511.50 |
|           6 | PVC Dumbbell Set 10 kg       |  1299.00 |       774.00 |
+-------------+------------------------------+----------+--------------+
```

Explanation: Compares each product with the average of *its own* category, highlighting premium items within every category.

Report 17 – EXISTS / NOT EXISTS: cart items a returning customer has never bought before

SQL query

```
SELECT c.customer id AS id, CONCAT(c.first name,' ',c.last name) AS customer,
       LEFT(p.product_name,28) AS product_in_cart, ci.quantity AS qty, p.price
FROM Customers c
JOIN Shopping Cart sc ON sc.customer id = c.customer id
JOIN Cart_Items ci    ON ci.cart_id = sc.cart_id
JOIN Products p       ON p.product_id = ci.product_id
WHERE EXISTS (SELECT 1 FROM Orders o WHERE o.customer id = c.customer id)
  AND NOT EXISTS (SELECT 1 FROM Orders o JOIN Order Items oi ON oi.order id = o.order id
                  WHERE o.customer_id = c.customer_id AND oi.product_id = ci.product_id)
ORDER BY c.customer id, p.product name
LIMIT 12;
```

Output

```
+----+--------------+------------------------------+-----+---------+
| id | customer     | product_in_cart              | qty | price   |
+----+--------------+------------------------------+-----+---------+
|  1 | Aarav Sharma | Atomic Habits                |   1 |  499.00 |
|  1 | Aarav Sharma | M331 Silent Plus Wireless Mo |   1 |  899.00 |
|  2 | Priya Nair   | 25W Fast Charger             |   1 | 1199.00 |
```

```
|  2 | Priya Nair   | Adilette Comfort Slides      |   1 | 1799.00 |
|  2 | Priya Nair   | The Psychology of Money      |   1 |  349.00 |
|  3 | Rohan Mehta  | PVC Dumbbell Set 10 kg       |   1 | 1299.00 |
|  3 | Rohan Mehta  | Women's Cotton Kurta         |   2 | 1899.00 |
|  5 | Vikram Singh | Induction Cooktop 2000W      |   2 | 2499.00 |
|  5 | Vikram Singh | Leather Formal Shoes         |   2 | 2299.00 |
|  6 | Sneha Reddy  | Leather Formal Shoes         |   2 | 2299.00 |
|  6 | Sneha Reddy  | Yoga Mat 6mm                 |   1 |  549.00 |
|  8 | Meera Joshi  | 511 Slim Fit Jeans           |   1 | 2799.00 |
+----+--------------+------------------------------+-----+---------+
```

Explanation: EXISTS tests whether a subquery returns any row. The report lists products waiting in the cart of customers who have ordered before but never bought that product – a basis for reminder or discount e-mails.

Report 18 – NOT EXISTS: purchased products not yet reviewed by the buyer

SQL query

```
SELECT o.customer_id, CONCAT(c.first_name,' ',c.last_name) AS customer, oi.product_id, LEFT(p.product_name,28) AS product
FROM Orders o
JOIN Customers c ON c.customer_id = o.customer_id
JOIN Order_Items oi ON oi.order_id = o.order_id
JOIN Products p ON p.product_id = oi.product_id
WHERE o.order_status = 'Delivered'
  AND NOT EXISTS (SELECT 1 FROM Reviews r WHERE r.customer id = o.customer id AND r.product id = oi.product id)
ORDER BY o.customer_id, oi.product_id
LIMIT 10;
```

Output

```
+-------------+---------------+------------+------------------------------+
| customer_id | customer      | product_id | product                      |
+-------------+---------------+------------+------------------------------+
|           1 | Aarav Sharma  |         11 | Leather Formal Shoes         |
|           3 | Rohan Mehta   |          2 | Rockerz 450 Bluetooth Headph |
|           3 | Rohan Mehta   |          7 | Men's Formal Shirt           |
|           3 | Rohan Mehta   |         14 | Induction Cooktop 2000W      |
|           7 | Karthik Rao   |          5 | ColorFit Pulse 3 Smartwatch  |
|           9 | Arjun Patel   |          8 | Round Neck Cotton T-Shirt    |
|          10 | Divya Menon   |          3 | 25W Fast Charger             |
|          10 | Divya Menon   |          3 | 25W Fast Charger             |
|          10 | Divya Menon   |         23 | Football Size 5              |
|          14 | Neha Kulkarni |          4 | M331 Silent Plus Wireless Mo |
+-------------+---------------+------------+------------------------------+
```

Explanation: Delivered purchases with no matching review – candidates for a “please rate your purchase” e-mail.

C. Combined and Summary Reports

Report 19 – Window function: product ranking by revenue inside each category

SQL query

```
SELECT category_name, product, revenue,
       RANK() OVER (PARTITION BY category_name ORDER BY revenue DESC) AS rank_in_category
FROM (SELECT cat.category name, LEFT(p.product name,26) AS product,
             SUM(oi.quantity * oi.unit price) AS revenue
      FROM Order_Items oi
      JOIN Orders o ON o.order_id = oi.order_id AND o.order_status <> 'Cancelled'
      JOIN Products p ON p.product id = oi.product id
      JOIN Categories cat ON cat.category id = p.category id
      GROUP BY cat.category_name, p.product_id, p.product_name) t
ORDER BY category_name, rank_in_category
LIMIT 14;
```

Output

```
+---------------+---------------------------+-----------+------------------+
| category_name | product                   | revenue   | rank_in_category |
+---------------+---------------------------+-----------+------------------+
| Books         | Wings of Fire             |   2093.00 |                1 |
| Books         | Atomic Habits             |   1996.00 |                2 |
| Books         | Database System Concepts  |   1798.00 |                3 |
| Books         | The Psychology of Money   |    349.00 |                4 |
| Clothing      | Men's Formal Shirt        |   5996.00 |                1 |
| Clothing      | Round Neck Cotton T-Shirt |   2397.00 |                2 |
| Clothing      | Women's Cotton Kurta      |   1899.00 |                3 |
| Electronics   | Redmi Note 13 5G (8GB/128G | 113754.00 |                1 |
| Electronics   | Rockerz 450 Bluetooth Head |   9234.00 |                2 |
| Electronics   | M331 Silent Plus Wireless |   6437.00 |                3 |
| Electronics   | ColorFit Pulse 3 Smartwatc |   5997.00 |                4 |
| Electronics   | 25W Fast Charger          |   5084.00 |                5 |
| Footwear      | Revolution 7 Running Shoes |  19975.00 |                1 |
| Footwear      | Leather Formal Shoes      |   4598.00 |                2 |
+---------------+---------------------------+-----------+------------------+
```

Explanation: A derived table totals revenue per product; RANK() OVER (PARTITION BY …) then ranks products separately inside every category without collapsing rows.

Report 20 – Customer 360 report: orders, spend, reviews and cart in one row each

SQL query

```
SELECT c.customer_id AS id, CONCAT(c.first_name,' ',c.last_name) AS customer,
       COALESCE(o.orders,0) AS orders, COALESCE(o.spent,0) AS spent,
       COALESCE(r.reviews,0) AS reviews, COALESCE(k.cart_items,0) AS cart_items
FROM Customers c
LEFT JOIN (SELECT customer_id, COUNT(*) AS orders, SUM(total_amount) AS spent
           FROM Orders WHERE order_status <> 'Cancelled' GROUP BY customer_id) o ON o.customer_id = c.customer_id
LEFT JOIN (SELECT customer_id, COUNT(*) AS reviews FROM Reviews GROUP BY customer_id) r ON r.customer_id = c.customer_id
LEFT JOIN (SELECT sc.customer_id, COUNT(*) AS cart_items FROM Shopping_Cart sc
           JOIN Cart_Items ci ON ci.cart_id = sc.cart_id GROUP BY sc.customer_id) k ON k.customer_id = c.customer_id
ORDER BY spent DESC;
```

Output

```
+----+--------------------+--------+----------+---------+-------------+
| id | customer           | orders | spent    | reviews | cart_items  |
+----+--------------------+--------+----------+---------+-------------+
| 14 | Neha Kulkarni      |      4 | 52611.00 |       5 |           2 |
|  3 | Rohan Mehta        |      4 | 36723.00 |       3 |           2 |
|  7 | Karthik Rao        |      4 | 35833.00 |       4 |           0 |
| 10 | Divya Menon        |      3 | 31560.00 |       4 |           2 |
|  2 | Priya Nair         |      1 | 19439.00 |       1 |           3 |
|  1 | Aarav Sharma       |      1 | 13844.00 |       3 |           3 |
| 13 | Siddharth Das      |      2 |  7493.00 |       1 |           2 |
| 12 | Pooja Bansal       |      1 |  7491.00 |       0 |           3 |
|  9 | Arjun Patel        |      2 |  7391.00 |       4 |           0 |
|  6 | Sneha Reddy        |      1 |  2499.00 |       0 |           2 |
|  8 | Meera Joshi        |      1 |  1499.00 |       0 |           3 |
|  5 | Vikram Singh       |      1 |  1499.00 |       0 |           2 |
| 16 | Lakshmi Narayanan  |      0 |     0.00 |       0 |           0 |
| 11 | Imran Khan         |      0 |     0.00 |       0 |           0 |
| 15 | Rahul Verma        |      0 |     0.00 |       0 |           0 |
|  4 | Ananya Iyer        |      0 |     0.00 |       0 |           0 |
+----+--------------------+--------+----------+---------+-------------+
```

Explanation: Four independent aggregates are joined to the customer list, giving customer-service staff a complete picture of every customer in a single row.

Report 21 – VIEW and executive summary

SQL query

```
CREATE OR REPLACE VIEW v order summary AS
SELECT o.order id, CONCAT(c.first name,' ',c.last name) AS customer, o.order date, o.order status,
       COUNT(oi.order_item_id) AS line_count, SUM(oi.quantity) AS units, o.total_amount,
       pay.payment_method, pay.payment_status
FROM Orders o
JOIN Customers c ON c.customer id = o.customer id
JOIN Order Items oi ON oi.order id = o.order id
JOIN Payments pay ON pay.order id = o.order id
GROUP BY o.order id, customer, o.order date, o.order status, o.total amount, pay.payment method, pay.payment status;


SELECT * FROM v_order_summary WHERE order_status = 'Placed';


SELECT (SELECT COUNT(*) FROM Customers) AS customers,
       (SELECT COUNT(*) FROM Products)  AS products,
       (SELECT COUNT(*) FROM Orders WHERE order_status <> 'Cancelled') AS active_orders,
       (SELECT SUM(total_amount) FROM Orders WHERE order_status = 'Delivered') AS delivered_revenue,
       (SELECT ROUND(AVG(rating),2) FROM Reviews) AS avg_rating;


DROP VIEW v_order_summary;
```

Output

```
+----------+---------------+---------------------+--------------+------------+-------+--------------+------------------+-------------
---+
| order_id | customer      | order_date          | order_status | line_count | units | total_amount | payment_method   |
payment_status |
+----------+---------------+---------------------+--------------+------------+-------+--------------+------------------+-------------
---+
|       25 | Sneha Reddy   | 2026-10-14 15:23:00 | Placed       |          1 |     1 |      2499.00 | Cash on Delivery | Pending
|
|       26 | Karthik Rao   | 2026-10-18 11:16:00 | Placed       |          3 |     4 |     22396.00 | UPI              | Successful
|
|       27 | Siddharth Das | 2026-10-19 19:31:00 | Placed       |          3 |     6 |      5994.00 | Cash on Delivery | Pending
|
|       28 | Karthik Rao   | 2026-10-20 15:32:00 | Placed       |          4 |     6 |      5194.00 | UPI              | Successful
|
+----------+---------------+---------------------+--------------+------------+-------+--------------+------------------+-------------
---+
+-----------+----------+---------------+-------------------+-------------+
| customers | products | active_orders | delivered_revenue | avg_rating  |
+-----------+----------+---------------+-------------------+-------------+
|        16 |       25 |            25 |         149013.00 |        4.16 |
+-----------+----------+---------------+-------------------+-------------+
```

Explanation: A view stores the four-table join under a name so it can be queried like a table; it is dropped at the end to leave the schema unchanged. The final query is a one-row dashboard built entirely from scalar subqueries.

D. Conclusion

Joins combined up to five tables to answer business questions about invoices, geography, couriers, payments, carts and product performance. Subqueries (scalar, IN, EXISTS, correlated and derived-table) compared individual rows with group-level benchmarks to identify high-value customers, premium products, abandoned carts and unreviewed purchases. Together these reports show that the normalized design from Assignments 3–6 still supports rich analysis: every question was answered with plain SQL and no duplicated data.